

Day4 实训手册：重复点赞 — 幂等设计

一、今日任务目标

目标	说明
理解幂等	同一请求执行 1 次和 N 次，结果完全相同
认识常见幂等手段	唯一索引、幂等 Token、乐观锁、状态机、去重表的适用场景
掌握 Redis 原子操作	用 Lua 脚本把"判断 + 写入 + 计数"合并为原子操作
掌握双重保障	Redis 原子判断 + 数据库唯一索引兜底
完成验证	1 用户 100 次并发点赞 = 1 条关系；混合负载压测后 SCARD 与计数始终一致
课堂实战	给"分享"加 Lua 幂等，并改造关注接口（第八节，课内启动、课下完成）

二、问题场景（先复现，再优化）

2.1 竞态窗口到底在哪：先纠正一个直觉误区

很多同学的第一反应是：100 个并发重复点赞，计数会 +100。**其实不会**——Day3 代码里 INCR 是被 SADD 的返回值“把门”的：

```
1 Boolean added = redisTemplate.opsForSet().add(...); // SADD 原子：100 个并发里恰好 1 个
   拿到 true
2 if (!Boolean.TRUE.equals(added)) return fail; // 其余 99 个在这里被拦下
3 redisTemplate.opsForValue().increment(...); // 只有那 1 个线程会 INCR
```

所以纯点赞压测永远是 SCARD=1、count=1，怎么压都一致。真正的竞态窗口在**点赞与取消点赞并发交错**时：线程 A 的 SADD 和 INCR 之间，插进了线程 B 完整的 SREM+DECR：

```
1 线程A(点赞): SADD → 1 (SCARD=1)
2 线程B(取消):          SREM → 1 (SCARD=0) → DECR → count=-1 → 代码钳位 SET 0
3 线程A(点赞): INCR → count=1
4 最终: SCARD=0, count=1 ❌ 永久漂移
```

课堂讨论一个反直觉的细节：Day3 代码里“计数不小于 0”的钳位保护 (count<0 则 SET 0)，反而把本应与 A 的 +1 互相抵消的瞬时 -1 “吃掉”了，把一次瞬时不一致变成永久漂移。
结论：补丁式防御救不了原子性缺失，必须用 Lua 从根上解决。

2.2 复现方法一：redis-cli 手动交错（100% 确定性，课堂演示首选）

不用碰运气，按顺序敲命令“人肉扮演”两个线程（先把 Day3 版本跑起来，笔记 9 需存在且未点赞，必要时 DEL like:9 like:count:9 清场）：

```
1 # — 扮演线程A（点赞），只执行第一步就“卡住” —
2 SADD like:9 6 # 返回 1, SCARD 此刻 = 1
3
```

```
4 # — 扮演线程B (取消点赞) , 趁窗口完整执行 —
5 SREM like:9 6 # 返回 1, SCARD 此刻 = 0
6 DECR like:count:9 # count: 0 → -1 (代码里接下来会钳位成 0)
7 SET like:count:9 0 # 模拟代码的下限保护
8
9 # — 线程A“恢复”, 执行第二步 —
10 INCR like:count:9 # count: 0 → 1
11
12 # 核对:
13 SCARD like:9 # 0
14 GET like:count:9 # 1 ❌ 不一致, 稳定复现!
```

2.3 复现方法二: JMeter 混合负载 (随机复现, 压测感强)

1. 测试计划结构 (一个线程组、两个取样器顺序执行, 每个线程循环“点赞→取消→点赞→取消...”):

```
1 测试计划
2   └─ 线程组: 线程数 100, Ramp-up 1 秒, 循环次数 50
3       └─ HTTP信息头管理器: X-User-Id: 6; Content-Type: application/json
4           └─ HTTP请求1: POST http://localhost:8080/api/notes/9/like
5               └─ HTTP请求2: DELETE http://localhost:8080/api/notes/9/like
```

- 2. 压测前清场: `DEL like:9 like:count:9;`
- 3. 跑完后核对: `SCARD like:9` 与 `GET like:count:9` 是否相等 (通常 count 比 SCARD 大 1 或几 = 竞态命中);
- 4. 多试几轮: 每轮都先 DEL 两个 Key 再重压; 窗口只有微秒级, 一轮没中很正常, 把循环次数加到 100~200 能提高命中率。

改完 Lua 版本 (第四、五节) 后, 用同一套混合负载再压一遍: SCARD 与 count 永远相等——前后对比就是今天最直观的验收。

三、理论基础：幂等与原子性

3.1 什么是幂等：来自真实事故的教训

幂等 (Idempotent) : 同一操作执行 1 次和 N 次, 对系统造成的影响完全相同。

重复请求从哪来? (全部在生产中真实发生过):

来源	场景
用户重复点击	双击点赞/提交按钮, 前端未禁用
客户端重试	网络抖动, App/前端自动重发请求
网关/代理重试	Nginx 超时后自动重发上游请求
消息队列重投	Day6 消费失败后 MQ 会重新投递
恶意脚本	并发刷赞、羊毛党

真实案例: 支付重复扣款、订单重复创建、短信重复发送 —— 都是幂等缺失的代价。

本课的点赞虽然“小”，但原理与支付防重完全一致。

3.2 HTTP 方法与幂等性（面试常考）

方法	是否幂等	说明
GET	是	只读，多次执行无副作用
PUT	是	整体覆盖写，多次执行结果相同
DELETE	是	删同一个资源，结果相同
POST	否	每次可能创建新资源 — 点赞接口是 POST，必须自己保证幂等

3.3 常见幂等方案对比（选型思维）

幂等方案看起来很多，本质上只有两类思路：要么**让重复的写入被拦下来**（唯一索引、去重表、幂等 Token），要么**让重复的执行产生相同结果**（状态机、乐观锁、原子操作）。下面逐一讲清每种方案的原理、常见做法和具体例子——它们都是生产系统的标配。

（1）唯一索引：让数据库当最后的裁判

原理：给业务字段加唯一约束（UNIQUE KEY），无论多少并发请求同时插入，数据库只允许一条成功，其余全部抛 DuplicateKeyException。应用层不需要自己协调，数据库底层的索引结构天然串行。

常见做法：给“能唯一标识一次业务动作”的字段组合建唯一索引（如“用户+笔记”），插入时捕获重复键异常并转为友好提示：

```
1 CREATE TABLE t_note_like (  
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3     user_id BIGINT NOT NULL,  
4     note_id BIGINT NOT NULL,  
5     create_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
6     UNIQUE KEY uk_user_note (user_id, note_id)  -- 同一用户对同一笔记只能有一条  
7 );
```

```
1 try {  
2     likeMapper.insert(new NoteLike(userId, noteId));  
3 } catch (DuplicateKeyException e) {  
4     return Result.fail("您已经点过赞了");    // 重复请求被索引拦截  
5 }
```

真实例子：基线项目的 `t_note_like` 就是这么做的（Day1 你见过）；订单表常用 `UNIQUE(order_no)` 防重复下单。

局限：它只能管住“这行数据重不重”，管不住旁边的计数有没有多加一次——所以它通常做**兜底**而不是第一道闸。

（2）幂等 Token：先领票，再办事

原理：把一次业务拆成两步——进入页面时先向服务器领一个**一次性令牌**；提交时带上令牌，服务端“校验 + 作废”一步原子完成，第二次提交同一令牌会被直接拒绝。重复请求根本进不了业务逻辑。

常见做法（电商下单页是教科书案例）：

```
1 ① 用户进入订单确认页 → 服务端生成 UUID, SET token:{uuid} 1 EX 300, 随页面下发
2 ② 用户点“提交订单” (携带 token) → 服务端用 Lua 脚本原子执行:
3     token 存在? → DEL 并放行, 执行下单
4     不存在?    → 直接返回“请勿重复提交”
5 ③ 手抖双击第二次提交 → token 已被删 → 被拒
```

真实例子：银行转账表单、电商下单确认页、一切“提交按钮不能被连点”的场景。

局限：每次业务多一轮“令牌”的往返，流程重一点这种高频轻量操作不适合它，但下单这种低频重操作它最稳妥。

(3) 乐观锁：更新时检查“数据还是不是我读到的那份”

原理：读取时不加锁（所以叫“乐观”——赌冲突很少发生）；表里加一个**版本号列**，更新时把“版本号必须等于我读到的那个”写进 WHERE 条件：匹配则更新成功且版本 +1；不匹配说明期间有人改过，本次更新影响行数为 0，业务方重新读取后重试或直接失败。

常见做法：

```
1 ALTER TABLE t_account ADD COLUMN version INT DEFAULT 0;
2
3 -- 扣款：只有版本号没变才成功
4 UPDATE t_account
5 SET balance = balance - 100, version = version + 1
6 WHERE id = 1 AND version = 666;
7 -- 影响行数 = 1: 更新成功
8 -- 影响行数 = 0: 数据被并发改过，需重读新版本后重试（或报“操作冲突”）
```

真实例子：账户余额并发修改、库存扣减。与悲观锁（SELECT ... FOR UPDATE 先锁行再改）的区别：悲观锁让所有并发请求排队，吞吐低；乐观锁不加锁，只有冲突方付出重试代价——冲突少时乐观锁明显更快。

(4) 状态机：状态只能往前走，不能回头

原理：业务对象有明确的状态流转图（如订单：待支付→已支付→已发货→已完成），任何状态变更必须声明“从哪个状态到哪个状态”。把“原状态”写进 WHERE 条件：第一次执行把状态从“待支付”改成“已支付”，重复请求再来时 status='待支付' 已匹配不上，影响行数为 0，自然被拦下。

常见做法：

```
1 -- 支付回调（可能被支付渠道重复通知）：
2 UPDATE t_order SET status = 'PAID', pay_time = NOW()
3 WHERE order_id = 1001 AND status = 'UNPAID';
4 -- 只有第一次影响 1 行；重复回调影响 0 行 → 幂等达成
```

真实例子：支付渠道重复通知、订单取消、退款审核——一切有“状态流转”的流程型业务。

(5) 去重表：用一张表记住“处理过哪些”

原理：单独建一张表，专门存已处理过的**业务唯一键**（消息 ID、请求流水号）。处理前先往去重表插一条（唯一索引拒重复），插入成功才执行业务；去重表的插入与业务操作放在**同一个数据库事务里**，保证要么都成功、要么都不发生。

常见做法：

```
1 CREATE TABLE t_msg_consume (
2     msg_id VARCHAR(64) PRIMARY KEY,      -- 消息的全局唯一 ID
3     consumed_at DATETIME DEFAULT CURRENT_TIMESTAMP
4 );
5
6 -- 消费者处理逻辑:
7 -- ① INSERT INTO t_msg_consume(msg_id) VALUES('msg-001');  ← 重复则插入失败, 说明处理过
8 -- ② 执行业务逻辑 (与①同事务)
```

真实例子：MQ 防重复消费。Day6 的点赞落库我们没有另建去重表——因为 `t_note_like` 自身的唯一索引本质上就是一张去重表。

(6) Redis 原子操作：高频场景的第一道闸（本课主方案）

原理：两点——① SADD / SETNX 这类命令的**返回值**天然表达“是不是第一次”（SADD 返回 1 = 本次新增，返回 0 = 已存在）；② Redis 单线程执行命令，单条命令天然原子，多步操作用 **Lua 脚本**合并成一个原子执行单元（3.5 详述）。

真实例子：本课主线一点赞/收藏/分享/关注全部用“Lua 脚本原子判断 + 计数”，微秒级完成拦截。

局限：Redis 是缓存，宕机可能丢数据，所以关键数据仍要数据库唯一索引兜底——这就是 4.3“双重保障”的由来。

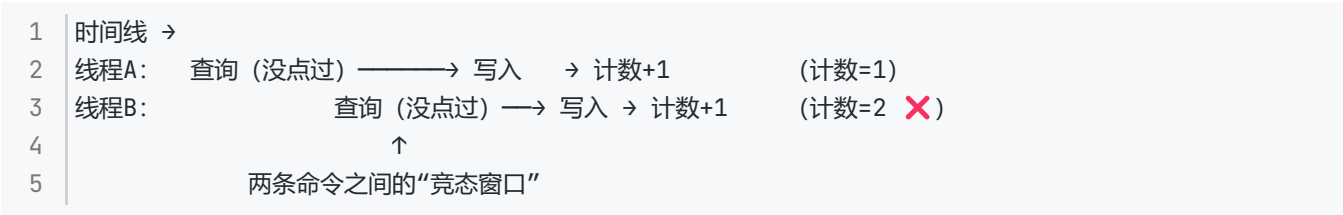
方案对比总表与选型口诀

方案	核心动作	延迟量级	适用场景	本课是否使用
唯一索引	数据库约束拒绝重复插入	毫秒级	关系类数据（点赞/关注/订单号）	✅ 兜底
幂等 Token	先领票再办事，用后作废	两次往返	表单提交/下单/支付	❌（太重）
乐观锁	更新时校验版本号	毫秒级	余额/库存等更新类	❌
状态机	原状态写进 WHERE 条件	毫秒级	订单等流程型业务	❌
去重表	唯一键表防重复消费	毫秒级	MQ 消息消费	思路同唯一索引
Redis 原子操作	返回值判断 / Lua 脚本	微秒级	高频轻量的状态操作	✅ 主方案

选型口诀：防“插入重复”→唯一索引；防“按钮连点”→幂等 Token；防“并发改同一行”→乐观锁；防“流程重复推进”→状态机；防“消息重复消费”→去重表；高频轻量操作→Redis 原子操作 + 唯一索引兜底。

面试常问：“如何设计一个幂等接口？”——先判断业务动作的类型（插入/更新/流转/消费），再选对应方案，最后加兜底，而不是背方案清单。

3.4 竞态条件：为什么"先查后写"一定会出事



关键认知: **synchronized** 只在单实例内有效，集群部署下失效；
而 Redis 单线程串行执行命令，**单条命令天然原子**，无需应用层加锁。
但多条命令之间仍不原子 — 所以今天的主角是 Lua 脚本。

对应到本课: Day3 Redis 版本的竞态窗口已经不在“查与写”之间 (SADD 自己把门)，
而是移到了 **SADD 与 INCR 之间**——第二节的交错演示抓的就是这个窗口。

3.5 Lua 脚本：把多步操作合成一次原子执行

(1) 什么是 Lua 脚本？为什么它能解决原子性？

Lua 是一种轻量级脚本语言，Redis **内置了 Lua 解释器**：你可以通过 **EVAL** 命令把一段脚本发给 Redis 服务器，Redis **会把整个脚本当作一条命令执行**——执行期间不会插入任何其他客户端的命令（单线程串行）。于是原本有竞态窗口的“判断 → 写入 → 计数”三步，变成一个不可分割的整体。

一句话: **Lua 脚本 = 把“几条命令”升级成“一条命令”**，而一条命令天然原子。

(2) 为什么不用 Redis 事务 (MULTI) ？

方案	能“读后判断”吗	往返次数	结论
MULTI/EXEC 事务	❌ 打包后不能根据中间结果分支	1 次	只能做固定序列，不能判断
WATCH 乐观锁	✅ 但写法繁琐，易被打断重试	多次	高并发下成功率低
EVAL Lua 脚本	✅ 条件判断、多命令写入都行	1 次	本课选择

(3) 常见写法：记住这个骨架

每个 Redis Lua 脚本都由四个要素组成：

```
1 | -- 要素① KEYS[]: 脚本要操作的所有 Key (由调用方传入，下标从 1 开始)
2 | -- 要素② ARGV[]: 额外参数 (如 userId)
3 | -- 要素③ redis.call('命令', ...): 在脚本里执行 Redis 命令
4 | -- 要素④ return: 返回值给调用方 (最常用数字，也可返回字符串/表)
```

三个模板，由浅入深：

```
1 | -- 模板1: 读后写 (判断 Key 是否存在，再决定写不写)
2 | local v = redis.call('GET', KEYS[1])
3 | if v then return 0 end          -- 已存在，什么都不做
4 | redis.call('SET', KEYS[1], ARGV[1])
5 | return 1
6 |
```

```

7  -- 模板2: 占坑式写入 (分布式锁的雏形, Day5 会遇到)
8  local ok = redis.call('SET', KEYS[1], ARGV[1], 'NX', 'EX', 10)
9  if ok then return 1 else return 0 end
10
11 -- 模板3: 判断 + 写入 + 计数 (本课要用的, 最经典)
12 local added = redis.call('SADD', KEYS[1], ARGV[1])  -- SADD 返回值: 1=新增 0=已存在
13 if added == 1 then
14     redis.call('INCR', KEYS[2])  -- 只有真正新增时才计数 +1
15     return 1
16 else
17     return 0
18 end

```

写脚本的三条纪律 (违反任何一条都可能造成生产事故) :

1. **必须短小**: 脚本执行期间所有其他命令都在排队, 脚本太长会卡住整个 Redis;
2. **所有 Key 通过 KEYS[] 传入**: 不要在脚本里硬编码 Key (集群环境要求 Key 显式声明才能路由到同一节点);
3. **禁止耗时操作**: 不写遍历大集合的循环、不调用 **KEYS *** 这类慢命令。

(4) 动手: 用 EVAL 亲自验证原子性 (上课必做)

理解了骨架, 现在把模板3真正跑一遍。进入容器: `docker exec -it xhs-redis redis-cli:`

```

1  # EVAL 脚本 参数: 脚本体 | Key个数 | Key... | 参数...
2  EVAL "local added = redis.call('SADD', KEYS[1], ARGV[1]) if added == 1 then
3      redis.call('INCR', KEYS[2]) return 1 else return 0 end" 2 demo:likes demo:count u1
4
5  # 返回 1 (点赞成功)
6
7  # 再执行一次, 模拟重复请求 → 返回 0, 且计数不变 (原子拦截)
8
9  # 验证:
10 SISMEMBER demo:likes u1          # 1
11 GET demo:count                  # 1
12
13 # 清理:
14 DEL demo:likes demo:count

```

课堂讨论: Lua 脚本在 Redis 中执行时会阻塞其他命令, 如果脚本写得很长很复杂会发生什么? (全部命令排队 → 脚本必须短小、禁止耗时操作, 即上面“三条纪律”的第 1 条。)

复盘提问: 为什么我们选 Lua 而不是 MULTI/WATCH? (答案在上面 (2) 的对比表里, 自己复述一遍)

四、方案设计

4.1 Day3 版本的竞态窗口在哪

注意：不是“两个线程都 SADD 成功”（SADD 原子，第二个必然返回 0），而是点赞的两步之间插进了取消的两步：

```
1 线程A(点赞): SADD → 1 (SCARD=1)
2 线程B(取消):          SREM → 1 (SCARD=0) → DECR → -1 → 钳位 SET 0
3 线程A(点赞): INCR → count=1
4 最终: SCARD=0, count=1 ❌ (第二节复现的就是它)
```

解决：把“SADD + 判断 + INCR”多步操作放进 Lua 脚本，Redis 单线程串行执行，整个脚本天然原子——别的命令再也插不进两步之间；取消同理（SREM + DECR 合为一个脚本）。

4.2 点赞幂等脚本

```
1 -- KEYS[1]=like:{noteId} KEYS[2]=like:count:{noteId} ARGV[1]=userId
2 local added = redis.call('SADD', KEYS[1], ARGV[1])
3 if added == 1 then
4     redis.call('INCR', KEYS[2])
5     return 1      -- 点赞成功
6 else
7     return 0      -- 已经点过，直接返回，什么都不改
8 end
```

4.3 双重保障

层	手段	作用
Redis	Lua 原子脚本	高性能拦截 99.9% 的重复请求
MySQL	UNIQUE(user_id, note_id)	最终兜底，Day6 落库时捕获 DuplicateKeyException

五、编码实现

按 `code/day4/` 目录完成：

序号	文件	操作
1	<code>service/InteractService.java</code>	【替换】点赞/取消、收藏/取消改为 Lua 脚本执行

要点：

```
1 private static final DefaultRedisScript<Long> LIKE_SCRIPT = new DefaultRedisScript<>(
2     "local added = redis.call('SADD', KEYS[1], ARGV[1]) " +
3     "if added == 1 then redis.call('INCR', KEYS[2]) return 1 else return 0 end",
4     Long.class);
5
6 Long result = redisTemplate.execute(LIKE_SCRIPT,
7     Arrays.asList(RedisKeys.like(noteId), RedisKeys.likeCount(noteId)),
8     userId.toString());
9 if (result == null || result == 0) {
10     return Result.fail("您已经点过赞了");
11 }
```

六、验证与压测

6.1 幂等验证（核心验收）

- 1. 重置环境: `DEL like:9 like:count:9` (选一篇未点赞的笔记) ;
- 2. JMeter: 100 线程、1 次循环、固定 `X-User-Id: 6`, 同时请求 `POST /api/notes/9/like` ;
- 3. 验收 (全部必须满足) :

```
1 SCARD like:9          = 1          (点赞关系只有1条)
2 GET  like:count:9     = 1          (计数只加1次)
3 100个响应中: 成功=1, "已经点过赞"=99
```

6.2 取消点赞与混合负载验证（呼应第二节）

- 1. 同样方式并发 100 次取消点赞, 确认计数不会变成负数;
- 2. 用第二节 2.3 的同一套 JMeter 混合负载 (POST+DELETE, 100 线程 × 50 循环) 重压 Lua 版本: 结束后 `SCARD like:9` 与 `GET like:count:9` 必须相等, 多跑几轮均相等—竞态窗口已被 Lua 封死。

七、结果记录表

测试项	预期	实际结果	是否通过
100并发点赞后关系数	1		
100并发点赞后计数	1		
重复请求响应	提示已点赞		
100并发取消后计数	0 (不为负)		
混合负载 (100线程×50循环) 后 SCARD 与 count	相等 (多轮均相等)		

八、课堂实战：把你的代码改成原子操作（自己动手）

今天老师带做的是点赞/收藏的 Lua 改造，现在轮到你把自己的代码升级。
任务分两步，第一步是基础，第二步是挑战。

任务一（必做）：给 Day3 的“分享”加幂等

你在 Day3 写的 `share` 方法还是“SADD → INCR”两条命令，存在竞态窗口。请把它改成 Lua 脚本：

```
1  -- KEYS[1]=share:{noteId} KEYS[2]=share:count:{noteId} ARGV[1]=userId
2  local added = redis.call('SADD', KEYS[1], ARGV[1])
3  if added == 1 then
4      redis.call('INCR', KEYS[2])
5      return 1
6  else
7      return 0
8  end
```

照着 `InteractService` 中点赞脚本的调用方式（`redisTemplate.execute(script, keys, args)`）替换 `share/unshare` 的实现即可。

验收：固定同一用户 100 并发调分享接口，`SCARD share:{id} = 1`、`GET share:count:{id} = 1`。

任务二（挑战）：关注接口幂等改造

关注接口 `POST /api/users/{id}/follow` 目前直接写 MySQL，高并发下同样会重复插入。请参照点赞思路：

- Key 设计：`follow:{userId}`（我关注的人，Set）+ `fans:count:{targetId}`（对方的粉丝数，计数器）；
- 用 Lua 脚本原子完成“SADD 关注集合 + INCR 粉丝数”，重复关注返回提示；
- 取消关注：SREM + DECR（防负数）；
- MySQL 写入交给 Day6（今天只写 Redis）。

验收标准：

任务一：100 并发分享，关系数与计数都为 1；

任务二：关注后 `SISMEMBER follow:{我} {对方} = 1`，`GET fans:count:{对方}` 加 1；

重复关注返回提示且不重复计数；

取消关注后再关注，计数正确不会错乱。

选做挑战：关注是双向关系（我关注了他 = 他多了个粉丝），如果 Redis 写了一半宕机，两个 Key 会不一致 —— 想想 Day6 的 MQ 能不能解决？

九、思考题

1. 为什么 Lua 脚本在 Redis 中是原子的？如果脚本执行时间太长会有什么问题？
2. 如果 Redis 挂了，幂等还靠什么保证？（提示：数据库唯一索引）
3. 对比“先查后写”和“Lua 脚本”，各自的一致性级别是什么？